AutoAlert

Antigravity Development Prompt Book

Use these prompts **in sequence**. Paste only one stage at a time into Antigravity. Do not move to the next stage until the current stage runs successfully, its tests pass, and you have reviewed the changes.

Prompt	Purpose
0	Project rules / SKILL.md
1	Foundation and repository setup
2	Video / CCTV input
3	OpenCV frame processing
4	Object detection
5	Object tracking
6	Vehicle collision candidate detection
7	Prolonged person motionlessness
8	Visual anomaly detection
9	Event verification engine
10	Evidence recording
11	Number plate detection
12	OCR and temporal aggregation
13	Incident object / package
14	FastAPI backend
15	Database
16	Central dashboard
17	Facility matching + simulated notifications
18	Testing and evaluation
19	Edge-device benchmarking
20	Deployment preparation

Target Project Structure

```
AutoAlert/
|-- .agents/
|   `-- skills/
|       `-- autoalert-engineering/
|           `-- SKILL.md
|-- .gitignore
|-- .env.example
|-- README.md
|-- requirements.txt
|-- requirements-dev.txt
|-- config/
|   |-- settings.yaml
|   |-- cameras.yaml
|   `-- facilities.yaml
|-- data/
|   |-- videos/
|   |   |-- raw/
|   |   `-- test/
|   |-- images/
|   |-- plates/
|   `-- test_cases/
|-- models/
|   |-- detection/
|   |-- tracking/
|   `-- license_plate/
|-- src/
|   |-- video/
|   |-- detection/
|   |-- tracking/
|   |-- events/
|   |-- license_plate/
|   |-- evidence/
|   |-- location/
|   |-- core/
|   `-- utils/
|-- backend/
|   |-- routes/
|   |-- database/
|   `-- schemas/
|-- dashboard/
|   |-- pages/
|   `-- components/
|-- tests/
|-- scripts/
|-- outputs/
|   |-- incidents/
|   |-- videos/
|   |-- frames/
|   `-- logs/
`-- docs/
```

This is the target architecture. Antigravity should create and implement it incrementally rather than filling every module with placeholder logic on the first run.

Prompt 0 - Create the AutoAlert Engineering Skill

Create **.agents/skills/autoalert-engineering/SKILL.md** and place the following content in it. This gives the coding agent persistent project-specific engineering constraints.

```
---
name: autoalert-engineering
description: Engineering rules for the AutoAlert AI/IoT emergency detection platform.
---

# AutoAlert Engineering Rules

AutoAlert upgrades existing CCTV infrastructure using an AI/IoT Box.
The laptop is the prototype representation of that future box.

Core flow:
Existing CCTV -> Video -> Frame Processing -> Detection -> Tracking ->
Event Candidate -> Verification -> Evidence -> Incident Package ->
Central Backend -> Dashboard / simulated emergency response.

Rules:
1. Implement one stage at a time.
2. Every major module must be independently testable.
3. Keep detection, tracking, event detection, verification, evidence and backend concerns separate.
4. Never claim a feature works unless executable code and validation exist.
5. Never invent accuracy, FPS, latency, cost or deployment metrics.
6. Do not treat bounding-box overlap as proof of collision.
7. Do not treat motionlessness as proof of injury.
8. Do not describe simple HSV/color segmentation as reliable blood detection.
9. Do not treat OCR output as ground truth.
10. OCR must support unreadable / low-confidence results.
11. Do not hard-code credentials, tokens, API keys or private URLs.
12. Do not claim real police, ambulance or hospital integration unless authorized integration exists.
13. Prototype emergency-service workflows must be simulated.
14. Prefer modular, explainable solutions over unnecessary complexity.
15. Do not rewrite unrelated working modules.
16. Before a major change: inspect -> plan -> implement -> test -> report.
17. After a change, state exactly what was implemented and what remains unimplemented.
18. Do not optimize for Raspberry Pi until the laptop pipeline is validated.
19. Preserve structured logging and configurable paths/settings.
20. Ask before downloading large/random datasets or introducing major new dependencies.

Collision candidate logic should eventually consider multiple temporal signals such as:
tracking, relative motion, trajectory changes, distance changes, sudden motion changes,
temporal persistence and scene context.

Number-plate pipeline:
vehicle detection -> tracking -> best-frame selection -> plate detection ->
crop/preprocess -> OCR -> temporal aggregation -> confidence.

Verified incident data may include:
incident ID, type, camera ID, timestamp, location, evidence frames,
short evidence video, object/track IDs, plate text when readable,
OCR confidence and verification status.
```

Prompt 1 - Foundation and Repository Setup

Paste this prompt only after the previous stage has been reviewed and validated.

```
You are the lead software engineer for AutoAlert.

Read .agents/skills/autoalert-engineering/SKILL.md before making changes.

AutoAlert is an AI/IoT emergency detection and response platform designed to
upgrade existing CCTV cameras rather than replace them.

Long-term architecture:
Existing CCTV -> AI/IoT Box -> Video Processing -> Object Detection ->
Tracking -> Event Detection -> Verification -> Evidence -> Incident Package ->
Central Backend -> nearest relevant facilities -> Dashboard / notifications.

The laptop currently represents the future AI/IoT Box.

Current target events:
1. Vehicle collision
2. Prolonged person motionlessness / possible emergency
3. Visual anomaly
4. Number plate detection and OCR

YOUR TASK IS ONLY STAGE 1.

1. Inspect the current workspace.
2. If empty, initialize the AutoAlert project foundation.
3. Create the target top-level directories:
   config, data, models, src, backend, dashboard, tests, scripts, outputs, docs.
4. Create Python package directories under src for:
   video, detection, tracking, events, license_plate, evidence, location, core, utils.
5. Create .gitignore, .env.example, README.md, requirements.txt and requirements-dev.txt.
6. The .gitignore must exclude .venv, __pycache__, *.pyc, .env,
   generated outputs, large generated video files, model weights where appropriate,
   and IDE temporary files.
7. Add only dependencies needed for the foundation. Do not install all future dependencies.
8. Create a minimal executable Python entry point.
9. Create a basic import/smoke test.
10. Create docs/development_plan.md containing Stages 1 through 20.
11. Do NOT implement YOLO, tracking, collision logic, OCR, FastAPI,
    dashboard or emergency-service integrations yet.
12. Do not download datasets or models.
13. Run validation/tests.
14. Report every file created, commands run, actual results, and what is still unimplemented.
15. Stop after Stage 1. Do not proceed automatically.
```

Prompt 2 - Video / CCTV Input

Paste this prompt only after the previous stage has been reviewed and validated.

```
Implement Stage 2: Video/CCTV Input.

Read the AutoAlert engineering skill first. Inspect the repository and explain
the implementation plan before coding.

Create a modular video source layer that can conceptually support:
- local MP4/video files
- webcam
- future RTSP CCTV streams

For this stage, prioritize local MP4 input.

Create or update:
src/video/camera.py
src/video/stream.py
tests/test_video.py

Requirements:
- open a video source
- validate successful initialization
- read frames
- expose useful source/frame metadata
- handle end-of-stream
- handle invalid sources cleanly
- release resources correctly
- avoid hard-coded absolute paths
- keep future RTSP support extensible

Do NOT implement object detection, tracking, collision detection, OCR or dashboard.
Do not download random videos. Add success and failure tests, run them, report
actual results and stop after Stage 2.
```

Prompt 3 - OpenCV Frame Processing

Paste this prompt only after the previous stage has been reviewed and validated.

```
Implement Stage 3: OpenCV Frame Processing.

Read the AutoAlert engineering skill and inspect the existing video module first.

Build a frame-processing layer:
Video Source -> Raw Frame -> Frame Processor -> Processed Frame.

Implement src/video/frame_processor.py and corresponding tests.

Support:
- frame numbering
- width and height metadata
- source FPS metadata where available
- timestamps derived appropriately from the video source
- configurable resizing
- optional basic preprocessing hooks
- validation for empty/invalid frames

Keep preprocessing modular and avoid unnecessary transformations.

Do NOT implement object detection, tracking, event detection or OCR.
Run tests, report actual results, and stop after Stage 3.
```

Prompt 4 - Object Detection

Paste this prompt only after the previous stage has been reviewed and validated.

```
Implement Stage 4: Object Detection.

Read the AutoAlert engineering skill first.

Integrate an Ultralytics object detector into the existing frame pipeline.

Initial classes of interest:
person, car, motorcycle, bus, truck.

Create or update:
src/detection/detector.py
src/detection/vehicle_detector.py
src/detection/person_detector.py
tests/test_detection.py

Requirements:
- keep model loading separate from video/frame processing
- return structured detection objects
- include class, confidence and bounding box
- make model configuration/path configurable
- handle model-loading/inference errors cleanly
- avoid hard-coded absolute paths
- do not claim accuracy or FPS unless actually measured

Do NOT implement tracking or collision detection yet.
If a valid model is available, run a small local inference test.
Report the model used, actual result, errors and any genuinely measured performance.
Stop after Stage 4.
```

Prompt 5 - Object Tracking

Paste this prompt only after the previous stage has been reviewed and validated.

```
Implement Stage 5: Object Tracking.

Read the engineering skill and inspect the detection output contract.

Add multi-object tracking so detected persons and vehicles can retain track IDs
across consecutive frames.

Create or update:
src/tracking/tracker.py
tests/test_tracking.py

Requirements:
- accept structured detections from Stage 4
- return track ID, class, confidence, bounding box and temporal history
- maintain enough trajectory history for future event logic
- handle tracks appearing/disappearing
- avoid coupling tracking to collision logic
- preserve clean interfaces

Do NOT decide whether an accident occurred.
Run tests on controlled input/video, report actual behavior, and stop after Stage 5.
```

Prompt 6 - Vehicle Collision Candidate Detection

Paste this prompt only after the previous stage has been reviewed and validated.

```
Implement Stage 6: Vehicle Collision Candidate Detection.

This stage produces CANDIDATE EVENTS only. It must not declare a verified accident.

Read the engineering skill first.

Create:
src/events/collision.py
tests/test_collision.py or extend tests/test_events.py

Use temporal tracking information. Do NOT implement:
if bounding_boxes_overlap: accident = True

Design an explainable candidate-scoring approach using multiple available signals,
such as relative distance change, trajectory change, abrupt motion change,
spatial interaction and temporal persistence.

Requirements:
- output a structured collision candidate
- include involved track IDs
- include signal/reason information
- include candidate confidence/score only if its meaning is clearly defined
- make thresholds configurable
- include negative test cases such as vehicles passing or stopping near each other
- keep verification separate

Do not invent model accuracy.
Run tests, explain limitations, and stop after Stage 6.
```

Prompt 7 - Prolonged Person Motionlessness

Paste this prompt only after the previous stage has been reviewed and validated.

```
Implement Stage 7: Prolonged Person Motionlessness Detection.

Read the engineering skill first.

Create:
src/events/motionless.py
appropriate tests

Using person tracks, detect prolonged low movement over a configurable time window.

Requirements:
- output 'possible emergency' / motionlessness candidate, not 'injured person'
- use temporal history rather than a single frame
- make movement and duration thresholds configurable
- reset state appropriately when movement resumes or a track disappears
- include positive and negative tests
- keep final verification separate

Do not infer injury, unconsciousness or medical state.
Run tests, report limitations, and stop after Stage 7.
```

Prompt 8 - Visual Anomaly Detection

Paste this prompt only after the previous stage has been reviewed and validated.

```
Implement Stage 8: Visual Anomaly Detection for the exhibition prototype.

Read the engineering skill first.

Create:
src/events/anomaly.py
appropriate tests

Implement a configurable OpenCV-based visual anomaly prototype using color/spatial
characteristics and temporal persistence.

Important:
- do NOT call this a reliable blood detector
- label output as a red/liquid-like visual anomaly or similar neutral description
- use configurable HSV/range settings rather than hard-coded assumptions where possible
- consider contour/area filtering and persistence
- return structured candidate information
- include obvious negative cases in tests

Document lighting, camera, compression and color ambiguity limitations.
Stop after Stage 8.
```

Prompt 9 - Event Verification Engine

Paste this prompt only after the previous stage has been reviewed and validated.

```
Implement Stage 9: Event Verification.

Read the engineering skill and inspect all candidate-event contracts.

Create:
src/events/verifier.py
src/core/event.py if needed
tests/test_events.py

Build a verification layer that receives candidate events and applies temporal,
confidence and evidence rules before creating a verified incident trigger.

Requirements:
- candidate event != verified incident
- configurable verification thresholds
- explicit verification states such as candidate, verifying, verified, rejected/expired
- avoid duplicate verification of the same ongoing event
- preserve reasons/signals used in the decision
- unit-test successful verification, rejection, expiry and duplicate suppression

Do not integrate backend notifications yet.
Stop after Stage 9.
```

Prompt 10 - Evidence Recording

Paste this prompt only after the previous stage has been reviewed and validated.

```
Implement Stage 10: Evidence Recording.

Read the engineering skill first.

Create:
src/evidence/recorder.py
src/evidence/frame_capture.py
src/evidence/incident_report.py
appropriate tests

For a verified event, support saving:
- key evidence frames
- a short incident video clip when enough buffered frames are available
- metadata describing evidence paths and timestamps

Requirements:
- use configurable output directories
- avoid overwriting unrelated incidents
- sanitize generated filenames
- handle write failures
- do not fabricate missing evidence
- keep evidence creation separate from OCR/backend logic

Run tests and a local controlled example if possible.
Stop after Stage 10.
```

Prompt 11 - Number Plate Detection

Paste this prompt only after the previous stage has been reviewed and validated.

```
Implement Stage 11: Number Plate Detection.

Read the engineering skill first.

Create:
src/license_plate/detector.py
src/license_plate/preprocessor.py
tests/test_plate_detection.py if appropriate

The pipeline should operate on relevant tracked vehicle frames and attempt to
locate a license-plate region.

Requirements:
- keep plate detection separate from OCR
- preserve vehicle track ID association
- support best-frame/crop selection hooks
- return 'not detected' rather than inventing a plate region
- make model/config paths configurable
- document whether a pretrained/custom detector is used
- do not silently download large models or datasets without approval

Run controlled tests and report actual limitations.
Stop after Stage 11.
```

Prompt 12 - OCR and Temporal Aggregation

Paste this prompt only after the previous stage has been reviewed and validated.

```
Implement Stage 12: Number Plate OCR.

Read the engineering skill first.

Create or update:
src/license_plate/ocr.py
src/license_plate/aggregator.py
tests/test_ocr.py

Use an OCR engine suitable for the prototype, such as PaddleOCR if selected and
compatible with the environment.

Pipeline:
plate crop -> preprocessing -> OCR -> normalization -> temporal aggregation -> confidence.

Requirements:
- OCR output is not ground truth
- associate readings with vehicle track IDs
- combine multiple frame readings when available
- return unreadable/low-confidence when evidence is insufficient
- do not fabricate missing characters
- preserve raw OCR result and normalized candidate where useful
- make Indian plate formatting validation a validation aid, not a mechanism that invents characters

Run tests and report actual OCR outputs on available controlled samples.
Stop after Stage 12.
```

Prompt 13 - Incident Object and Package

Paste this prompt only after the previous stage has been reviewed and validated.

```
Implement Stage 13: Incident Object and Incident Package.

Read the engineering skill first.

Create or update:
src/core/incident.py
src/evidence/incident_report.py
tests/test_incident.py

Define a structured incident contract containing fields such as:
- incident_id
- incident_type
- camera_id
- timestamp
- location if configured/available
- verification status
- involved track IDs
- evidence frames/video paths
- plate readings only when available
- confidence/reason metadata

Requirements:
- use typed models/dataclasses/Pydantic only if appropriate to current dependencies
- validate required fields
- do not fabricate GPS or OCR data
- support serialization suitable for future backend transmission
- ensure deterministic testability

Do not implement real notifications yet.
Stop after Stage 13.
```

Prompt 14 - FastAPI Backend

Paste this prompt only after the previous stage has been reviewed and validated.

```
Implement Stage 14: Central FastAPI Backend.

Read the engineering skill first.

Create:
backend/main.py
backend/routes/incidents.py
backend/routes/cameras.py
backend/routes/facilities.py
backend/schemas/ as needed
backend tests as appropriate

Requirements:
- health endpoint
- endpoint to receive validated incident packages
- endpoints to retrieve incidents/cameras/facilities as appropriate
- request validation
- clear HTTP errors
- no real emergency-service integrations
- no hard-coded credentials
- keep persistence behind a clean interface for Stage 15

Run the API tests and report actual results.
Stop after Stage 14.
```

Prompt 15 - Database

Paste this prompt only after the previous stage has been reviewed and validated.

```
Implement Stage 15: Database Persistence.

Read the engineering skill first and preserve the existing API contract.

Use SQLite for the prototype with a clean path to future PostgreSQL migration.

Create/update:
backend/database/database.py
backend/database/models.py
backend/database/repositories.py
database tests

Persist:
- incidents
- cameras
- simulated facilities
- relevant status/history fields

Requirements:
- avoid database logic inside route handlers
- handle initialization cleanly
- do not hard-code secrets
- use migrations only if justified for the current prototype
- test create/read/update flows
- do not add unnecessary infrastructure

Stop after Stage 15.
```

Prompt 16 - Central Dashboard

Paste this prompt only after the previous stage has been reviewed and validated.

```
Implement Stage 16: AutoAlert Command Center Dashboard.

Read the engineering skill first.

Use Streamlit for the prototype unless the existing repository has an approved
alternative.

Create/update:
dashboard/app.py
dashboard/pages/
dashboard/components/

Show:
- incident list
- incident details
- verification/status
- camera ID and timestamp
- evidence preview where available
- plate text only when available
- facility information
- map/location only when valid coordinates exist

Requirements:
- never fabricate missing data
- clearly mark simulated/demo data
- no real emergency notifications
- keep UI separate from backend logic

Run the dashboard locally and validate the main workflow.
Stop after Stage 16.
```

Prompt 17 - Facility Matching and Simulated Notifications

Paste this prompt only after the previous stage has been reviewed and validated.

```
Implement Stage 17: Facility Matching and SIMULATED Notifications.

Read the engineering skill first.

Use configured/demo facility records for:
- ambulance/EMS
- police station
- hospital

Requirements:
- match relevant nearby facilities when valid incident coordinates exist
- document the distance calculation used
- create simulated notification/acknowledgement states
- visibly label all notifications as prototype/simulated
- do not call real police, ambulance, hospital, SMS or emergency APIs
- support states such as pending, notified, acknowledged, responding where appropriate
- test matching and status transitions

Stop after Stage 17.
```

Prompt 18 - Testing and Evaluation

Paste this prompt only after the previous stage has been reviewed and validated.

```
Implement Stage 18: System Testing and Evaluation.

Do not add new major product features.

Read the engineering skill and audit the current repository.

Create a reproducible evaluation plan covering:
- video ingestion failures
- detection outputs
- tracking continuity
- collision candidate false positives/negatives on available controlled cases
- motionlessness cases
- anomaly limitations
- verification logic
- evidence recording
- plate detection/OCR
- API/database
- dashboard workflow

Measure only metrics that can actually be measured from available data.
Clearly separate:
- measured result
- unmeasured requirement
- known limitation

Add integration tests where appropriate.
Produce docs/evaluation.md with commands and actual results.
Stop after Stage 18.
```

Prompt 19 - Edge Device Benchmarking

Paste this prompt only after the previous stage has been reviewed and validated.

```
Implement Stage 19: Edge-Deployment Benchmark Preparation.

The current laptop prototype must remain working.

Read the engineering skill first.

Create scripts/benchmark.py and docs/edge_benchmark.md.

Benchmark the existing pipeline on the CURRENT machine for measurable values such as:
- input resolution
- model used
- inference time
- end-to-end processing time
- approximate FPS
- CPU/RAM/GPU usage where reliably measurable

Do not claim Raspberry Pi performance without running on Raspberry Pi.
Do not select final hardware from guesses.

Document what must later be measured on candidate edge hardware:
compute compatibility, model export/runtime, FPS, thermal behavior, power,
storage, network, camera/RTSP compatibility and recovery behavior.

Stop after Stage 19.
```

Prompt 20 - Deployment Preparation

Paste this prompt only after the previous stage has been reviewed and validated.

```
Implement Stage 20: Deployment Preparation for the AutoAlert prototype.

Read the engineering skill first.

Do not pretend this is a certified production emergency system.

Create/update:
docs/deployment.md
configuration examples
startup/run scripts where appropriate

Document:
- environment setup
- dependency installation
- configuration
- local video demo
- backend startup
- dashboard startup
- output/evidence directories
- database initialization
- logging
- troubleshooting
- security considerations
- limitations
- future edge-device deployment path

Ensure secrets remain outside source control.
Run a clean-start validation using the documented steps where feasible.
Report any unresolved issues.

Do not enable real emergency-service integrations.
Finish with a clear summary of the complete prototype state.
```

Recommended Working Rule

Do not paste all prompts at once. Start with Prompt 0, then Prompt 1. After Antigravity finishes a stage, review its changed files and test output. Only then paste the next numbered prompt. If a stage fails, fix that stage before continuing.

For the college prototype, the most important milestone is not reaching Prompt 20 quickly; it is having a traceable pipeline where each module can be demonstrated and its limitations can be explained.